

Reviewer Pack — Zeckendorf Length of NW Design Rows Bounds PRG Parity Bias

Entry b133863cfef2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 13:16:57 UTC

Zeckendorf Length of NW Design Rows Bounds PRG Parity Bias

Entry ID: b133863cfef2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 13:16:57 UTC

1. Conjecture

Field A (mathematical branch): Zeckendorf / Ostrowski numeration: representation of integers as sums of non-consecutive Fibonacci numbers and the resulting digit-length statistic $z(N)$ (Lekkerkerker-Carlitz metric theory of Fibonacci expansions, rarely deployed outside number theory) **Field B** (complexity object): Output bias of the Nisan-Wigderson pseudorandom generator NW_D^f against parity (linear χ_T) distinguishers, where D is a combinatorial (d,m,k,t) -design and f is a Boolean hard function

Statement:

Let $D = \{S_1, \dots, S_m\}$ be any (d,m,k,t) -NW design on $[d]$, and let z_i be the number of Fibonacci summands in the Zeckendorf representation of $N_i = \sum_{j \in S_i} 2^{j-1}$; set $Z(D) = (1/m) \sum_i z_i$. Then for every Boolean $f: \{0,1\}^k \rightarrow \{0,1\}$ whose largest non-empty Fourier coefficient $\max_{\emptyset \neq U \subseteq [k]} |\hat{f}(U)|$ equals η , the NW-PRG output bias against any non-empty parity test obeys $|\text{bias}_T(D,f)| := |2^{-d} \sum_{s \in \{0,1\}^d} \{\oplus_{i \in T} f(s|_{S_i})\}| \leq \eta^{\frac{1}{|T|}} \cdot 2^{\frac{1}{2} \cdot (|T| \text{ choose } 2)} \cdot (1 + Z(D)/k)^{|T|}$. A single (D,f,T) with $k \leq 5$, $m \leq 8$, $d \leq 10$ violating this inequality refutes the conjecture.

Rationale (proposer's reasoning):

NW analyses bound bias by $\eta^{\frac{1}{|T|}} \cdot 2^{\frac{1}{2} \cdot (|T| \text{ choose } 2)}$ (the standard Nisan-Wigderson lemma); the new factor $(1 + Z(D)/k)^{|T|}$ predicts that designs whose row-indicators are arithmetically 'simple' in the Fibonacci base (low Zeckendorf length, i.e., concentrated bit patterns) leak more correlation through f , while rows with maximally spread Zeckendorf decompositions act more like Beatty-equidistributed indices and force tighter cancellation. This bridges Zeckendorf metric theory with NW pseudorandomness — a pairing with effectively no prior literature — and offers a falsifier-friendly tightening of the classical bound that is meaningful only when $Z(D)$ is non-trivially correlated with the design's discrepancy.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 009844d175041d8f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 30 designs per (k,m) with $k \in \{3,4,5\}$, $m \in \{4..8\}$, $t \in \{1,2\}$, $d=2k$, 20 Boolean f per design with $\eta \in \{0.25, 0.5\}$, and all non-empty $T \subseteq [m]$ with $|T| \leq 3$: compute exact bias_T and check $|\text{bias}_T| \leq \eta^{|T| \cdot 2} \{t \cdot C(|T|, 2)\} \cdot (1 + Z(D)/k)^{-1}$. Support iff $\geq 95\%$ of triples satisfy and zero strict violations with $k \leq 5, m \leq 8, d \leq 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Nisan-Wigderson pseudorandom generator parity Fourier bias design - Zeckendorf representation Fibonacci digit sum pseudorandom generator - NW generator combinatorial design hard function correlation linear test

Top relevant hits considered: - [http://arxiv.org/abs/1311.6531v3] Brains and pseudorandom generators - [http://arxiv.org/abs/2404.12647v1] Simple constructions of linear-depth t -designs and pseudorandom unitaries - [http://arxiv.org/abs/2601.03730v1] Perception-Aware Bias Detection for Query Suggestions - [http://arxiv.org/abs/1906.10962v3] The Fibonacci Sequence and Schreier-Zeckendorf Sets - [http://arxiv.org/abs/1302.6583v1] A note on closed-form representation of Fibonacci numbers using Fibonacci trees - [http://arxiv.org/abs/2009.09510v1] Bounds on Zeckendorf Games - [http://arxiv.org/abs/1805.12210v2] Design of a general scientific CCD simulation and test system - [http://arxiv.org/abs/2301.08134v2] A Benchmark Generator for Combinatorial Testing

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def fibonacci(n):
    fib = [0, 1]
    for i in range(2, n+1):
        fib.append(fib[-1] + fib[-2])
    return fib

def zeckendorf_length(n):
    if n == 0:
        return 0
    fib = fibonacci(n)
    length = 0
```

```

while n > 0:
    for i in range(len(fib)-1, -1, -1):
        if fib[i] <= n:
            n -= fib[i]
            length += 1
            break
    return length

def generate_design(d, m, k, t):
    design = []
    for _ in range(m):
        row = [0]*d
        indices = random.sample(range(d), k)
        for i in indices:
            row[i] = 1
        design.append(row)
    return design

def compute_fourier_coefficient(f, n):
    max_coeff = 0
    for U in range(1, 2**n):
        coeff = 0
        for i in range(n):
            if U & (1 << i):
                coeff += f[i]
        max_coeff = max(max_coeff, abs(coeff))
    return max_coeff

def compute_bias(d, design, f, T):
    bias = 0
    for s in range(2**d):
        parity = sum(s >> i & 1 for i in T) % 2
        bias += (-1)**parity * f[s]
    return abs(bias / (2**d))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    k_values = [3, 4, 5]
    m_values = [4, 5, 6, 7, 8]
    t_values = [1, 2]
    eta_values = [0.25, 0.5]
    T_values = [set(T) for T in combinations(range(1, 6), 1)] + \
                [set(T) for T in combinations(range(1, 6), 2)]

    results = []
    for k in k_values:
        for m in m_values:
            for t in t_values:
                d = 2 * k
                design = generate_design(d, m, k, t)
                f_values = [random.choices([0, 1], k=2**d) for _ in range(20)]
                for f in f_values:
                    max_coeff = compute_fourier_coefficient(f, d)
                    if max_coeff != eta_values[0] and max_coeff != eta_values[1]:
                        continue

```

```

        for T in T_values:
            bias = compute_bias(d, design, f, T)
            results.append({
                "metric_name": "bias",
                "metric_value": bias,
                "instances_tested": 1,
                "conjecture_holds": abs(bias) <= eta_values[0]**len(T) * 2**(t*math.co
                "counterexample": ""
            })

    total_metric_value = sum(result["metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "seed": seed,
        "mean_metric_value": total_metric_value / len(results),
        "std_metric_value": 0, # Not computing std for simplicity
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(result["mean_metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.95) / len(re

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0 support_fraction={s
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE support_fraction too low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6b011f3.py", line 110, in <module>
 result = run_trial(seed)

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6b011f3.py", line 95, in run_trial
 support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ZeroDivisionError before any trial data was collected, so neither the support nor falsification criterion was evaluated. | next: Fix the trial generation so designs are actually produced (ensure results list is non-empty), then rerun the pre-registered sweep over $k \in \{3,4,5\}$, $m \in \{4..8\}$, $t \in \{1,2\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	86100
2	preregistration	claude_max	opus	0	0	7866
3	novelty	claude_max	opus	0	0	3636
4	novelty	claude_max	opus	0	0	8161
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13961
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12032
7	judge	claude_max	opus	0	0	10717

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142473 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/b133863cfef2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b133863cfef2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b133863cfef2.tar.gz` (if generated)